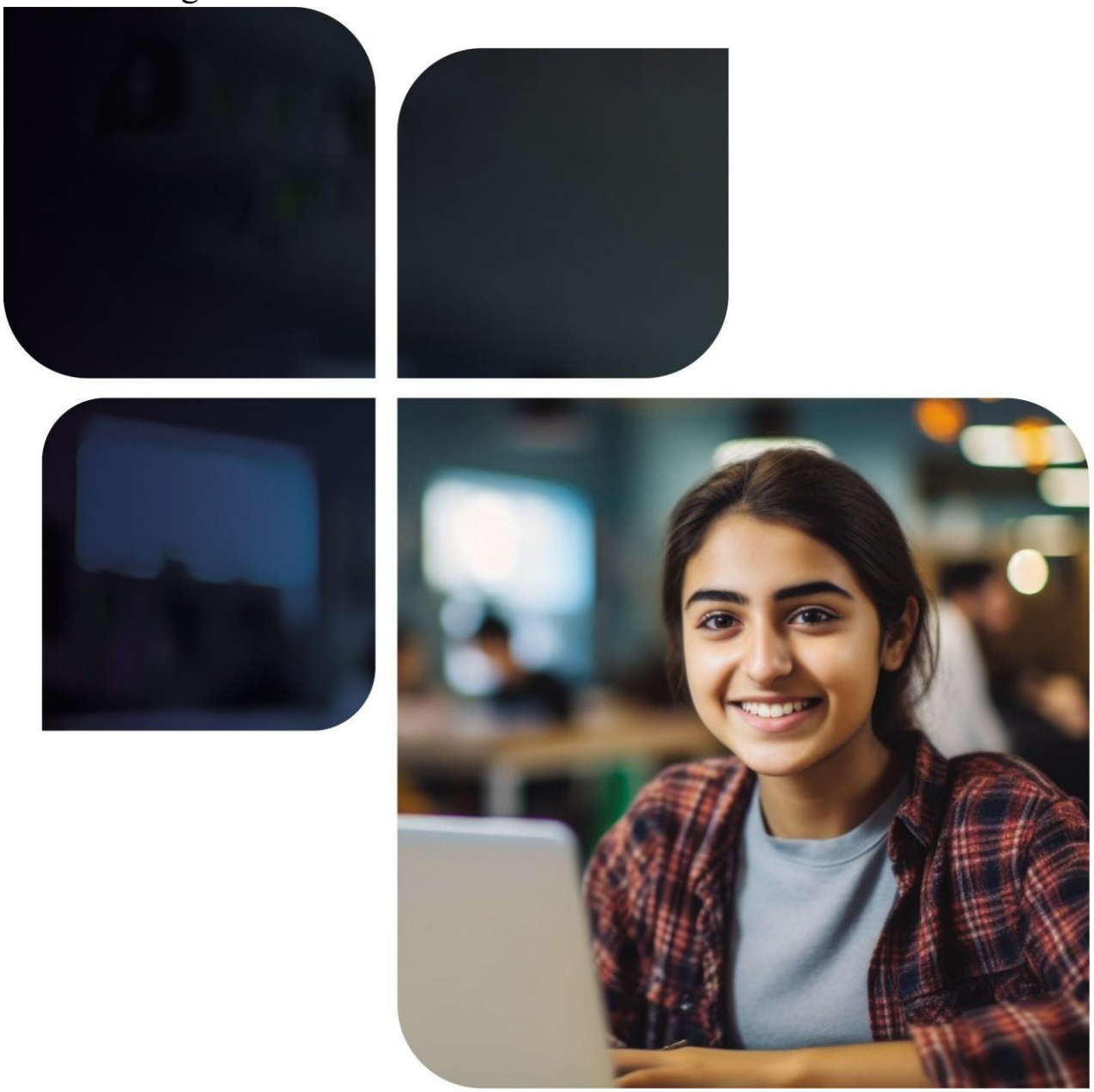
GLOBAL QUEST
T EC H N O LOG I ES

An ISO 9001 & ISO 21001
Certified Organizaton



# The Quest
# for your Dream Job
# Ends Here!!

# Quiz Game Project Report

*A report submitted in partial fulfillment of the requirements for the Organization*


**Name : YALLA PAVANI**

**Organization : Global Quest Technologies**

**Mentor : Mr. Bheemesh Raghupathy**


GLOBAL QUEST
TECHNOLOGIES


**Global Quest Technologies Training and Placement organization**

**Yelahanka New Town, Bengaluru**

**560064**

# PROJECT-1

## Abstract:

The project titled “Java-Based Quiz Program” was developed during an internship at Global Quest Technologies with the aim of creating a simple interactive quiz application using core Java concepts. This console-based program works like a real quiz game where the system displays multiple-choice questions (MCQs) to the user and the user selects an answer. The program then checks the answer and immediately shows whether it is correct or wrong, making the quiz interactive and easy to use. To make the quiz more interesting, the application includes features such as lifelines, safe levels, and a reward-based progress system, which give the user a better experience while playing the quiz. The main purpose of this project is to improve basic programming skills such as using loops, conditional statements, arrays, and handling user input in Java. It also helps in understanding how to organize code properly so that it is clear, structured, and easy to maintain. Overall, this project shows how Java can be used to develop simple and interactive console-based applications.


## Project Overview:

This Java-based quiz application is a terminal-driven program designed to give users an engaging and competitive quiz experience. It contains a predefined set of 10 questions, and each question provides four answer options for the user to choose from. The user moves through the quiz by selecting answers for each question, and the program checks whether the selected answer is correct or not. If the user selects a wrong answer, the quiz ends immediately, which makes the application more challenging and interactive. The program also includes a reward system where users earn virtual money for every correct answer, and it also provides safe levels that help the user secure a minimum amount of winnings during the quiz.

## Objective:

- To design an interactive quiz program using core Java.
- To handle user input during the quiz.
- To create a game-show style quiz in a console application.
- To use programming concepts like arrays, loops, and conditional statements effectively.
- To make the quiz more interesting by adding lifelines and simple visual feedback.

## Key Features & Functionalities:

### 1. Question Flow & Game Logic

- The quiz program displays **10 multiple-choice questions** to the user.
- Each question contains **four answer options**.
- The user must select the **correct answer** to move to the next question.
- If the answer is **correct**, the game continues to the next level.
- If the answer is **wrong**, the **game ends immediately**.
- The quiz includes **safe levels at Question 5 and Question 7**.
- After reaching a safe level, the user secures a **minimum reward amount** even if a later answer is incorrect.

## 2.User Interaction & Visual Enhancement

- Uses ANSI color codes for better readability.
- Displays messages in different colors for correct, wrong, and informational outputs.
- Provides a user-friendly console interface despite being non-GUI.

## 3.Technologies Used:

- **Programming Language:** JAVA
- **Concepts Used:**

- **Arrays (1D & 2D):** Used to store quiz questions, options, and answers in an organized way.
- **Loops (for loop):** Used to display questions repeatedly and control the flow of the quiz.
- **Conditional Statements (if-else, switch-case):** Used to check the user's answer and decide whether the game continues or ends.
- **Object-Oriented Concepts (basic structure):** Used to organize the program using classes and methods.
- **Random Class:** Used to generate random values for features like the audience poll simulation.
- **Scanner Class:** Used to take input from the user through the keyboard

## Project Description:

The “Java-Based Quiz Program” is a console-based interactive application developed using core Java concepts. The main objective of this project is to create a quiz game similar to real-world quiz shows where users answer multiple-choice questions and progress in the game based on their performance. The application contains 10 predefined questions, and each question has four answer options from which the user must select the correct one. If the user selects the correct answer, the quiz continues to the next question, but if a wrong answer is selected, the game ends immediately, which makes the quiz more challenging and interesting. The program also includes a reward system where users earn virtual money for each correct answer, and the reward amount increases as the user progresses in the game. In addition, the quiz has safe levels at Question 5 and Question 7, which ensure that the user secures a minimum reward amount even if they answer a later question incorrectly.

To make the quiz more interactive and strategic, the application provides three lifelines: the 50-50 lifeline, which removes two incorrect options; the Audience Poll, which shows percentage-based hints for each option; and the Skip Question lifeline, which allows the user to skip a question without losing progress. Each lifeline can be used only once during the entire quiz. The program uses the Scanner class to take user input and applies arrays, loops, and conditional statements to display questions, check answers, and control the flow of the game. The Random class is used to generate values for the audience poll, making the quiz more realistic.

Additionally, ANSI color codes are used to display messages in different colors to improve readability and enhance the overall user experience in the console. Overall, this project demonstrates the practical use of Java programming concepts such as arrays, loops, conditions, and user input handling, while also helping to develop logical thinking and problem-solving skills.

# SOURCE CODE:

```
import java.util.Scanner;
 public class quizapp
{
 static final String RESET = "\u001B[0m";
 static final String RED = "\u001B[31m";
 static final String GREEN = "\u001B[32m";
 static final String YELLOW = "\u001B[33m";
 static final String BLUE = "\u001B[34m";
 static final String CYAN = "\u001B[36m";
 static final String PURPLE = "\u001B[35m";

 public static void main(String[] args)
 {
    Scanner sc = new Scanner(System.in);

    String questions[] =
 {
          "Which company originally developed Java?",
          "Which keyword is used to define a class in Java?",
          "Which data type is used to store decimal numbers in Java?",
          "Which symbol is used for single line comments in Java?",
          "Which package contains the Scanner class?",
          "Which loop is best when the number of iterations is known?",
          "Which keyword is used to stop a loop in Java?",
          "Which concept allows one class to acquire properties of another?",
          "Which method is used to print output in Java?",
          "Which operator is used to add two numbers?"
    };

    String options[][] =
{
          {"Sun Microsystems", "Microsoft", "Google", "Apple"},
          {"class", "define", "create", "object"},
          {"int", "float", "boolean", "char"},
          {"//", "/* */", "#", "<!-- -->"},
          {"java.lang", "java.util", "java.io", "java.net"},
          {"while loop", "do while loop", "for loop", "switch"},
          {"break", "continue", "exit", "return"},
          {"Polymorphism", "Encapsulation", "Inheritance", "Abstraction"},
          {"System.print()", "System.println()", "print()", "println()"},
          {"+", "-", "*", "/"}
    };

    int answers[] = {0,0,1,0,1,2,0,2,1,0};
```

```
        int rewardArray[] =
{0,1000,2000,3000,4000,5000,6000,7000,8000,9000,10000};

        int prize = 0;
        int choice;

        boolean used5050 = false;
        boolean usedAudience = false;
        boolean usedSkip = false;
        boolean use5050now = false;

        System.out.println(PURPLE + "--------------------------------");
        System.out.println("         GQT QUIZ GAME         ");
        System.out.println("--------------------------------" + RESET);

        System.out.println(YELLOW + "Answer correctly to win money!");
        System.out.println("Use lifelines wisely!" + RESET);

        for(int i = 0; i < questions.length; i++) {

            System.out.println(CYAN + "\n-------------------------------------");
            System.out.println("Question " + (i + 1) + ": " + questions[i]);
            System.out.println("-------------------------------------" + RESET);

            int wrongOption = (answers[i] + 2) % 4;

            for(int j = 0; j < options[i].length; j++) {

                if(use5050now) {
                    if(j == answers[i] || j == wrongOption) {
                        System.out.println(BLUE + (j+1) + ". " + options[i][j] + RESET);
                    }
                }
                else {
                    System.out.println(BLUE + (j+1) + ". " + options[i][j] + RESET);
                }
            }

            System.out.println(YELLOW + "5. Lifeline");
            System.out.println("6. Quit" + RESET);

            System.out.print("Enter your choice: ");
            choice = sc.nextInt();

            if(choice < 1 || choice > 6) {
                System.out.println(RED + "Invalid choice! Try again." + RESET);
                i--;
                continue;
```

```
            }

            if(choice == 6)
        {
                System.out.println(RED + "You quit the game." + RESET);
                break;
            }

            if(choice == 5)
            {

                if(i >= 8) {
                    System.out.println(RED + "Lifelines are not applicable for 9 and 10
questions!" + RESET);
                    i--;
                    continue;
                }

                System.out.println(PURPLE + "Choose Lifeline:" + RESET);

                if(!used5050)
                    System.out.println("1. 50-50");
                else
                    System.out.println("1. 50-50 (Already Used)");

                if(!usedAudience)
                    System.out.println("2. Audience Poll");
                else
                    System.out.println("2. Audience Poll (Already Used)");

                if(!usedSkip)
                    System.out.println("3. Skip");
                else
                    System.out.println("3. Skip (Already Used)");

                int life = sc.nextInt();

                if(life == 1) {

                    if(!used5050) {
                        used5050 = true;
                        use5050now = true;
                        System.out.println(YELLOW + "50-50 activated!" + RESET);
                    }
                    else {
                        System.out.println(RED + "50-50 already used!" + RESET);
                    }
```

```
                }
                else if(life == 2) {

                    if(!usedAudience) {
                        usedAudience = true;
                        System.out.println(YELLOW + "Audience suggests option: " +
(answers[i] + 1) + RESET);
                    }
                    else {
                        System.out.println(RED + "Audience Poll already used!" + RESET);
                    }

                }
                else if(life == 3) {

                    if(!usedSkip) {
                        usedSkip = true;
                        System.out.println(YELLOW + "Question skipped!" + RESET);
                        continue;
                    }
                    else {
                        System.out.println(RED + "Skip already used!" + RESET);
                    }

                }
                else {
                    System.out.println(RED + "Invalid lifeline choice!" + RESET);
                }

                i--;
                continue;
            }

            if(choice - 1 == answers[i]) {

                System.out.println(GREEN + "Correct Answer!" + RESET);
                prize = rewardArray[i+1];
                System.out.println(GREEN + "You won: ₹" + prize + RESET);

            }
            else {

                System.out.println(RED + "Wrong Answer!" + RESET);
                System.out.println(GREEN + "Correct Answer is: " + (answers[i] + 1) + ".
" + options[i][answers[i]] + RESET);

                if(i < 5)
                    prize = 0;
```

```
            else if(i < 7)
                prize = 5000;
            else
                prize = 7000;

            System.out.println(RED + "You take home: ₹" + prize + RESET);
            break;
        }

        use5050now = false;
    }

    System.out.println(CYAN + "\n-----------------------------------");
    System.out.println("           GAME OVER              ");
    System.out.println("Final Prize: ₹" + prize);
    System.out.println("-----------------------------------" + RESET);
  }
}
```

# OUTPUT:

## //Display the conformation output:

```
Problems  @ Javadoc  Declaration  Console ×  Eclipse IDE for Enterprise Java and Web Developers 2
quizapp [Java Application] C:\Users\PAVANI\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.
-----------------------------------
            GQT QUIZ GAME
-----------------------------------
Answer correctly to win money!
Use lifelines wisely!

---------------------------------------
Question 1: Which company originally developed Java?
---------------------------------------
1. Sun Microsystems
2. Microsoft
3. Google
4. Apple
5. Lifeline
6. Quit
Enter your choice:
```

## //Display the first option and won money:

```
quizapp [Java Application] C:\Users\PAVANI\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.10.v20260205-0638\jre\bin\javaw.exe

-----------------------------------
Question 1: Which company originally developed Java?
-----------------------------------
1. Sun Microsystems
2. Microsoft
3. Google
4. Apple
5. Lifeline
6. Quit
Enter your choice: 1
Correct Answer!
You won: ₹1000

-----------------------------------
Question 2: Which keyword is used to define a class in Java?
-----------------------------------
1. class
2. define
3. create
4. object
5. Lifeline
6. Quit
Enter your choice:
```

**//Choose lifeline 50-50:**

```
Question 2: Which keyword is used to define a class in Java?
------------------------------------
1. class
2. define
3. create
4. object
5. Lifeline
6. Quit
Enter your choice: 5
Choose Lifeline:
1. 50-50
2. Audience Poll
3. Skip
1
50-50 activated!
```

**//Life  line 50-50 output:**

```
------------------------------------
Question 2: Which keyword is used to define a class in Java?
------------------------------------
1. class
3. create
5. Lifeline
6. Quit
Enter your choice: 1
Correct Answer!
You won: ₹2000
```

**//Life line  with audience poll**

```
Question 3: Which data type is used to store decimal numbers in Java?
------------------------------------
1. int
2. float
3. boolean
4. char
5. Lifeline
6. Quit
Enter your choice: 5
Choose Lifeline:
1. 50-50 (Already Used)
2. Audience Poll
3. Skip
2
Audience suggests option: 2
```

## //Audience poll output:

```
Question 3: Which data type is used to store decimal numbers in Java?
-------------------------------------
1. int
2. float
3. boolean
4. char
5. Lifeline
6. Quit
Enter your choice: 2
Correct Answer!
You won: ₹3000
```

## //50-50 chance error:

```
-------------------------------------
Question 4: Which symbol is used for single line comments in Java?
-------------------------------------
1. //
2. /* */
3. #
4. <!-- -->
5. Lifeline
6. Quit
Enter your choice: 5
Choose Lifeline:
1. 50-50 (Already Used)
2. Audience Poll (Already Used)
3. Skip
1
50-50 already used!
```

## //Audience poll error

```
-------------------------------------
Question 4: Which symbol is used for single line comments in Java?
-------------------------------------
1. //
2. /* */
3. #
4. <!-- -->
5. Lifeline
6. Quit
Enter your choice: 5
Choose Lifeline:
1. 50-50 (Already Used)
2. Audience Poll (Already Used)
3. Skip
2
Audience Poll already used!
```

**//Click wrong answer**

```
-----------------------------------
Question 6: Which loop is best when the number of iterations is known?
-----------------------------------
1. while loop
2. do while loop
3. for loop
4. switch
5. Lifeline
6. Quit
Enter your choice: 2
Wrong Answer!
Correct Answer is: 3. for loop
You take home: ₹5000
```

**//Skipped Question**

```
-----------------------------------
Question 7: Which keyword is used to stop a loop in Java?
-----------------------------------
1. break
2. continue
3. exit
4. return
5. Lifeline
6. Quit
Enter your choice: 5
Choose Lifeline:
1. 50-50 (Already Used)
2. Audience Poll (Already Used)
3. Skip
3
Question skipped!
```

**//Game over**

```
-------------------------------------
Question 6: Which loop is best when the number of iterations is known?
-------------------------------------
1. while loop
2. do while loop
3. for loop
4. switch
5. Lifeline
6. Quit
Enter your choice: 2
Wrong Answer!
Correct Answer is: 3. for loop
You take home: ₹5000

-----------------------------------
            GAME OVER
Final Prize: ₹5000
-----------------------------------
```

# CONCLUSION:

The Java-Based Quiz Program is a simple console application developed using core Java concepts. This project mainly uses arrays, loops, conditional statements, and the Scanner class to take input from the user. The program asks multiple-choice questions and allows the user to select the correct answer. After each question, the program checks the answer and gives immediate feedback. If the answer is correct, the user can continue to the next question, and if the answer is wrong, the quiz ends. This makes the program interactive and interesting for the user.

The program also includes special features called lifelines such as 50-50, Audience Poll, and Skip Question. These lifelines help the user answer difficult questions and make the quiz similar to real quiz games. ANSI color codes are also used in the program to display different colors for messages like correct answers, wrong answers, and instructions. This makes the output clear and more attractive in the console.

By completing this project, we improved our understanding of Java programming and learned how to use different programming concepts together to build a real application. It also helped in developing problem-solving skills and logical thinking. In the future, this project can be improved by adding a graphical user interface (GUI), connecting it with a database to store questions, and adding more advanced features to make the quiz application more powerful and user-friendl

# PROJECT-2

# ABSTRACT (Swing Version):

The project "Java-Based Quiz Program using Swing" was developed during an internship at Global Quest Technologies. The main goal of this project was to create an interactive and user-friendly quiz application using Java GUI (Graphical User Interface) components. Unlike a normal console-based quiz, this application uses windows, buttons, and other GUI features to make it more interesting and easy to use.

The program shows multiple-choice questions (MCQs) in a window and allows the user to select answers. It gives instant feedback after each question and uses event-driven programming to respond to user actions. The quiz also includes lifelines, safe levels, and a reward system, which makes it similar to real quiz shows.

This project helped in learning core Java programming as well as GUI development skills, like using components, handling events, and designing user-friendly interfaces. Overall, it is a practical example of how Java Swing can be used to build interactive desktop applications.


## Project Overview:

This Java Swing-based quiz application is a program with a graphical user interface (GUI) that makes the quiz more fun and interactive. Unlike the console version, it uses windows, buttons, and dialog boxes for the user to interact with the quiz.
The quiz has 10 fixed questions, and each question has four options shown as radio buttons. Users can click on their answers, and the program checks them immediately.
The application also has a reward system where users earn virtual money for each correct answer. It includes safe levels to make sure users keep some minimum points, making the game more interesting and realistic.

## Objective

- To design an interactive quiz system using Java Swing.
- To implement event-driven programming for user interaction.
- To develop a user-friendly graphical interface.
- To simulate a real-time quiz game environment.
- To apply Java concepts such as arrays, conditions, and GUI components effectively.

## Technologies Used:

- **Programming Language:** Java
- **GUI Framework:** Java Swing

**Concepts Used:**

o Arrays (1D & 2D)
o Event Handling (ActionListener)
o Loops (for loop)
o Conditional statements (if-else, switch-case)
o Object-Oriented Programming
o Random Class (for Audience Poll)
o Swing Components (JFrame, JButton, JLabel, JRadioButton, JOptionPane)

## Project Description (Swing Version):

The project “Java-Based Quiz Program using Swing” is a graphical user interface (GUI) quiz application developed using core Java and Swing components. Its main purpose is to provide an interactive and visually appealing quiz experience similar to real quiz shows. The program includes ten predefined multiple-choice questions, each displayed in a window where users can select their answers using radio buttons and submit them with buttons. The answers are checked immediately, and feedback is shown through popup dialog boxes. A special feature of this quiz is the reward system, where users earn virtual money for each correct answer, and safe levels are included at Question 5 and Question 7 to ensure users keep a minimum amount even if they answer wrong after these points. To make the quiz more interesting and strategic, three lifelines are included: 50-50, which removes two wrong options; Audience Poll, which gives percentage-based hints through a popup; and Skip Question, which allows users to move to the next question without losing points. Each lifeline can be used only once, adding a decision-making element to the game. The application is designed using event-driven programming, so actions like button clicks trigger specific functions. Java Swing components such as JFrame, JButton, JLabel, JRadioButton, and JOptionPane are used to build the interface, making it easy to use and visually attractive compared to the console version. The program also includes a start confirmation and a quit option with confirmation to improve usability and control. Overall, this project demonstrates the practical use of Java Swing, event handling, and core Java programming concepts, enhances user experience, and helps in developing problem-solving and GUI development skills, making it a valuable and interactive extension of a console-based quiz application.

## SOURCE CODE:

```
package com.GQT.JavaBasics.project2;
	import javax.swing.*;
	import java.awt.event.*;
		public class quizappswings2 extends JFrame implements ActionListener {

			JLabel questionLabel, prizeLabel;
			JRadioButton op1, op2, op3, op4;
			JButton submitBtn, lifelineBtn, quitBtn;

			ButtonGroup group;

			int index = 0;
			int prize = 0;

			boolean used5050 = false;
			boolean usedAudience = false;
			boolean usedSkip = false;

			String questions[] = {
				"Which company originally developed Java?",
				"Which keyword is used to define a class in Java?",
				"Which data type is used to store decimal numbers in Java?",
				"Which symbol is used for single line comments in Java?",
				"Which package contains the Scanner class?",
				"Which loop is best when the number of iterations is known?",
				"Which keyword is used to stop a loop in Java?",
				"Which concept allows one class to acquire properties of another?",
				"Which method is used to print output in Java?",
				"Which operator is used to add two numbers?"
			};

			String options[][] = {
				{"Sun Microsystems", "Microsoft", "Google", "Apple"},
				{"class", "define", "create", "object"},
				{"int", "float", "boolean", "char"},
				{"//", "/* */", "#", "<!-- -->"},
				{"java.lang", "java.util", "java.io", "java.net"},
				{"while loop", "do while loop", "for loop", "switch"},
				{"break", "continue", "exit", "return"},
				{"Polymorphism", "Encapsulation", "Inheritance", "Abstraction"},
				{"System.print()", "System.println()", "print()", "println()"},
				{"+", "-", "*", "/"}
			};

			int answers[] = {0,0,1,0,1,2,0,2,1,0};
```

```
        int rewardArray[] =
{0,1000,2000,3000,4000,5000,6000,7000,8000,9000,10000};

        quizappswings2() {

            setTitle("GQT Quiz Game");
            setSize(600,400);
            setLayout(null);
            setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

            questionLabel = new JLabel();
            questionLabel.setBounds(50,30,500,30);
            add(questionLabel);

            prizeLabel = new JLabel("Prize: ₹0");
            prizeLabel.setBounds(450,10,120,30);
            add(prizeLabel);

            op1 = new JRadioButton();
            op1.setBounds(50,80,400,30);

            op2 = new JRadioButton();
            op2.setBounds(50,120,400,30);

            op3 = new JRadioButton();
            op3.setBounds(50,160,400,30);

            op4 = new JRadioButton();
            op4.setBounds(50,200,400,30);

            add(op1);
            add(op2);
            add(op3);
            add(op4);

            group = new ButtonGroup();
            group.add(op1);
            group.add(op2);
            group.add(op3);
            group.add(op4);

            submitBtn = new JButton("Submit");
            submitBtn.setBounds(50,260,100,30);
            submitBtn.addActionListener(this);
            add(submitBtn);

            lifelineBtn = new JButton("Lifeline");
            lifelineBtn.setBounds(200,260,100,30);
```

```
        lifelineBtn.addActionListener(this);
        add(lifelineBtn);

        quitBtn = new JButton("Quit");
        quitBtn.setBounds(350,260,100,30);
        quitBtn.addActionListener(this);
        add(quitBtn);

        loadQuestion();

        setVisible(true);
    }

    void loadQuestion() {

        if(index < questions.length) {

            questionLabel.setText("Q" + (index+1) + ": " + questions[index]);

            op1.setText(options[index][0]);
            op2.setText(options[index][1]);
            op3.setText(options[index][2]);
            op4.setText(options[index][3]);

            op1.setVisible(true);
            op2.setVisible(true);
            op3.setVisible(true);
            op4.setVisible(true);

            group.clearSelection();
        }
        else {

            JOptionPane.showMessageDialog(this,"GAME OVER\nFinal Prize:
₹"+prize);
            System.exit(0);
        }
    }

    int getSelected() {

        if(op1.isSelected()) return 0;
        if(op2.isSelected()) return 1;
        if(op3.isSelected()) return 2;
        if(op4.isSelected()) return 3;

        return -1;
    }
```

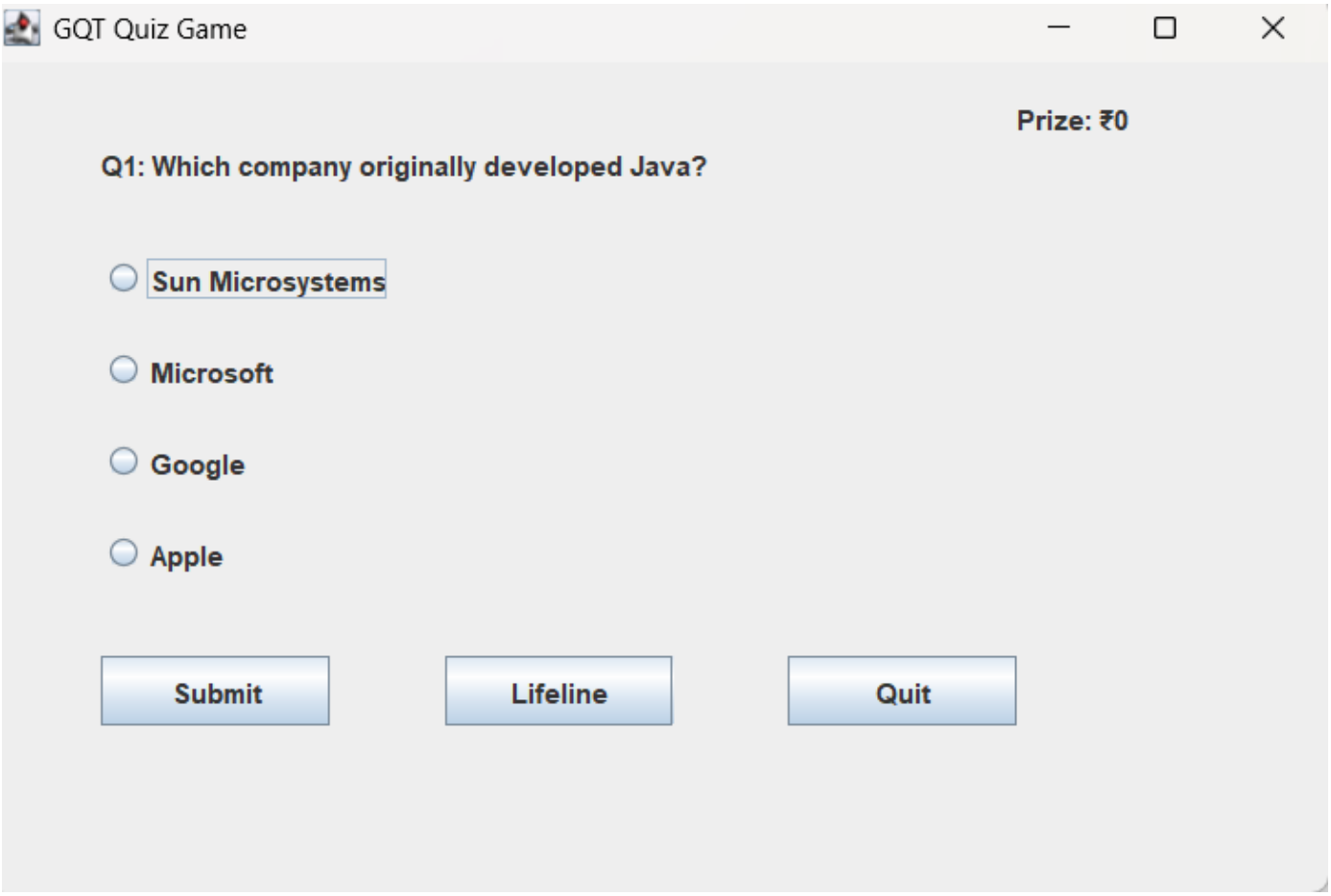
```
public void actionPerformed(ActionEvent e) {

    if(e.getSource() == submitBtn) {

        int selected = getSelected();

        if(selected == -1) {
            JOptionPane.showMessageDialog(this,"Please select an option");
            return;
        }

        if(selected == answers[index]) {

            prize = rewardArray[index+1];
            JOptionPane.showMessageDialog(this,"Correct Answer!\nYou won
₹"+prize);
            prizeLabel.setText("Prize: ₹"+prize);

            index++;
            loadQuestion();
        }
        else {

            JOptionPane.showMessageDialog(this,
                    "Wrong Answer!\nCorrect Answer:
"+options[index][answers[index]]
                            +"\nYou take home ₹"+prize);

            System.exit(0);
        }
    }

    if(e.getSource() == quitBtn) {

        JOptionPane.showMessageDialog(this,"You quit the game.\nPrize:
₹"+prize);
        System.exit(0);
    }

    if(e.getSource() == lifelineBtn) {

        String choice = JOptionPane.showInputDialog(
                "Choose Lifeline:\n1. 50-50\n2. Audience Poll\n3. Skip");

        if(choice == null) return;

        int life = Integer.parseInt(choice);
```

```
                if(life == 1) {

                    if(!used5050) {

                        used5050 = true;

                        int correct = answers[index];
                        int wrong = (correct + 1) % 4;

                        op1.setVisible(false);
                        op2.setVisible(false);
                        op3.setVisible(false);
                        op4.setVisible(false);

                        if(correct == 0 || wrong == 0) op1.setVisible(true);
                        if(correct == 1 || wrong == 1) op2.setVisible(true);
                        if(correct == 2 || wrong == 2) op3.setVisible(true);
                        if(correct == 3 || wrong == 3) op4.setVisible(true);

                        JOptionPane.showMessageDialog(this,"50-50 activated");

                    } else {

                        JOptionPane.showMessageDialog(this,"50-50 already used");
                    }
                }

                if(life == 2) {

                    if(!usedAudience) {

                        usedAudience = true;

                        JOptionPane.showMessageDialog(this,
                                "Audience suggests option: "+(answers[index]+1));

                    } else {

                        JOptionPane.showMessageDialog(this,"Audience Poll already
used");
                    }
                }

                if(life == 3) {

                    if(!usedSkip) {
```

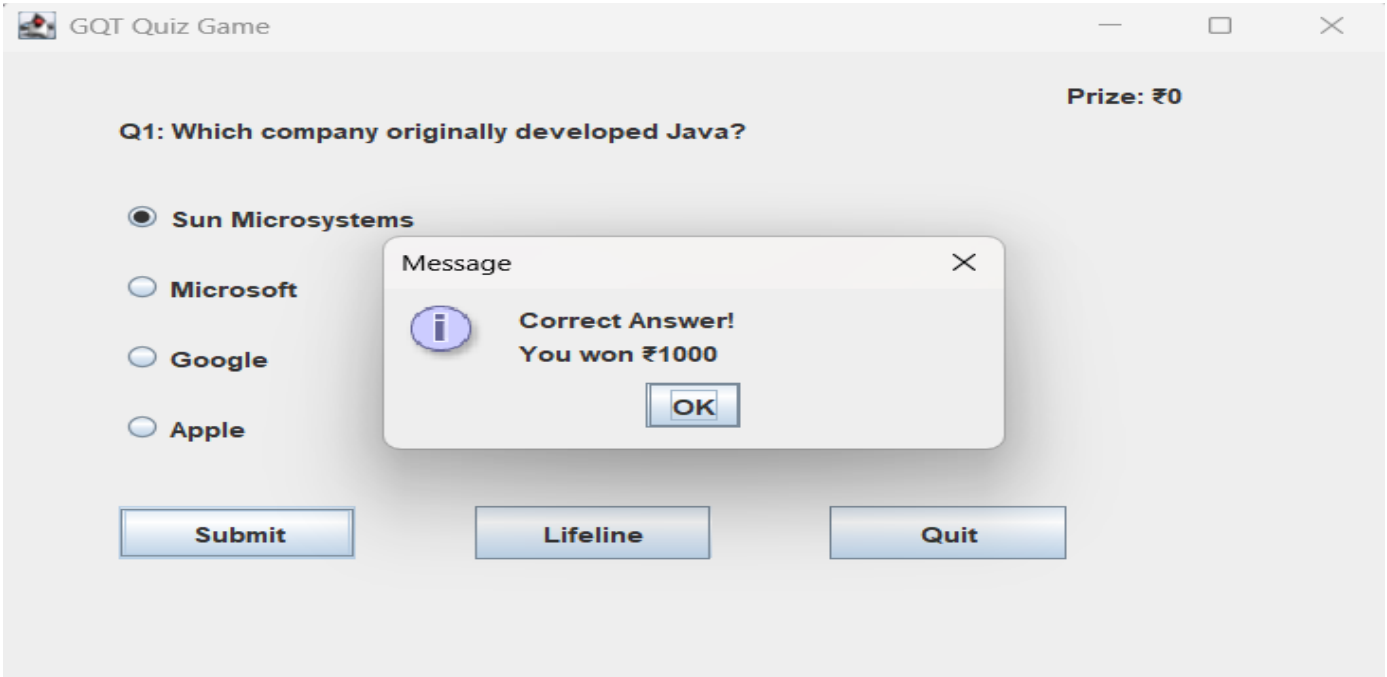
```
                usedSkip = true;

                JOptionPane.showMessageDialog(this,"Question Skipped");

                index++;
                loadQuestion();

            } else {

                JOptionPane.showMessageDialog(this,"Skip already used");
            }
        }
    }
}

public static void main(String[] args) {

    new quizappswings2();
}
}
```

**OUTPUT:**
**Starting Screens**

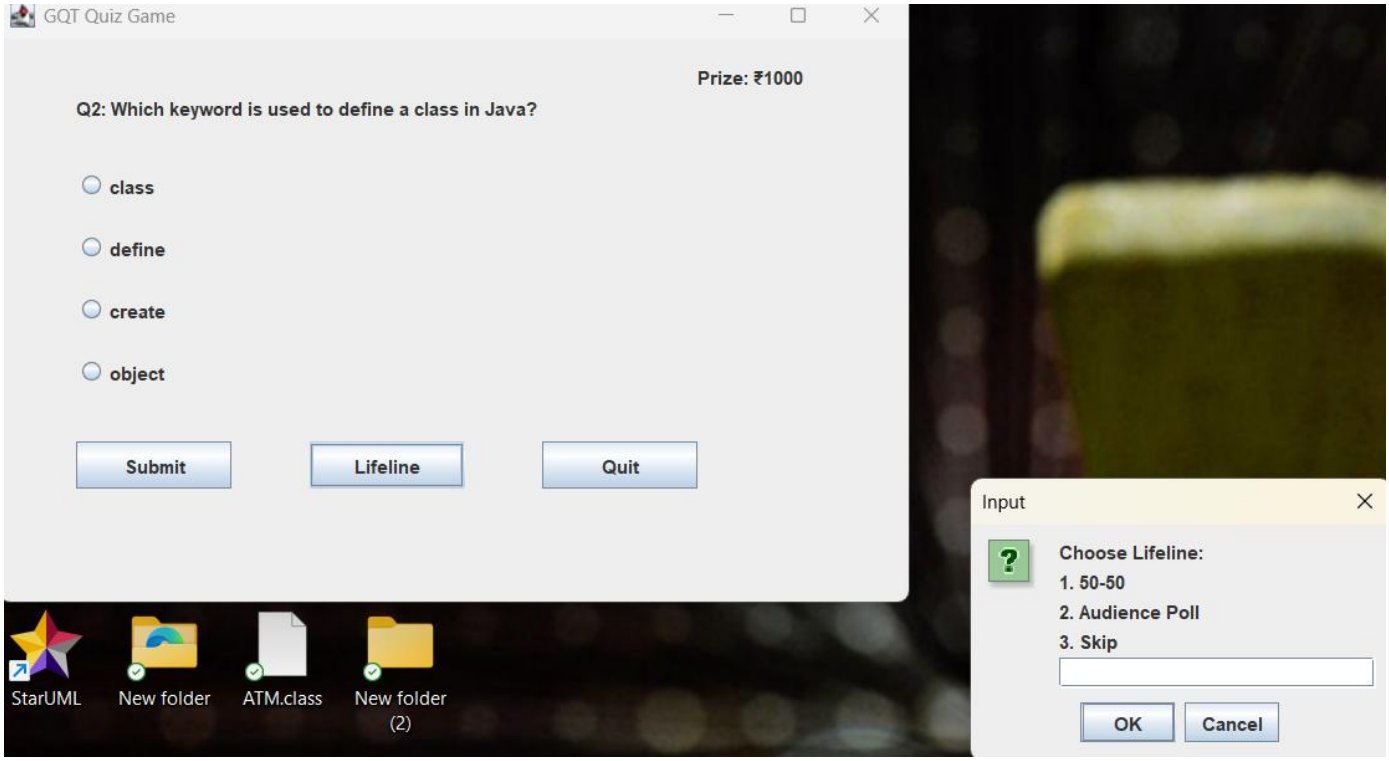
## //First Question Displayed



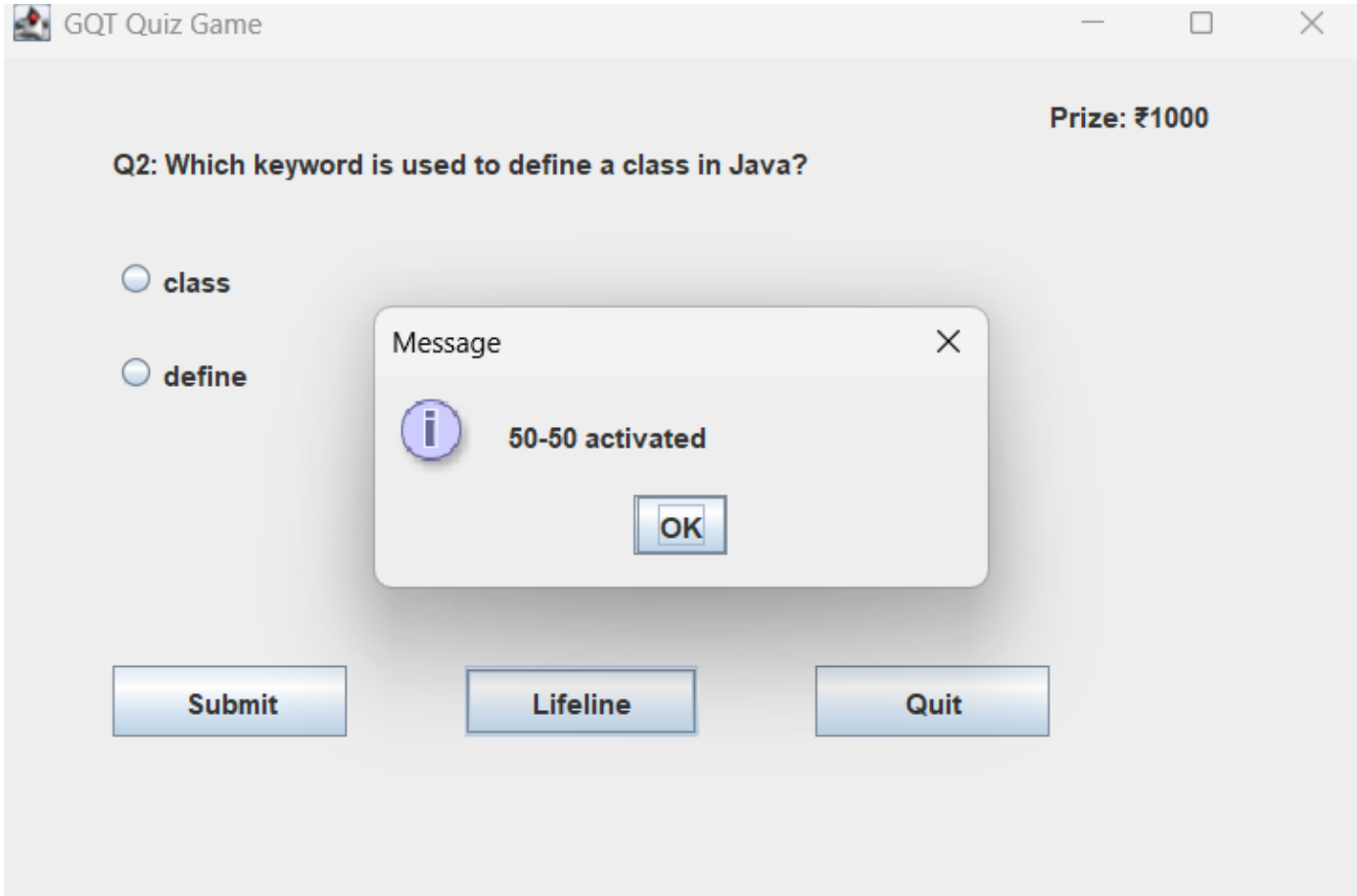

## //Choose The Option



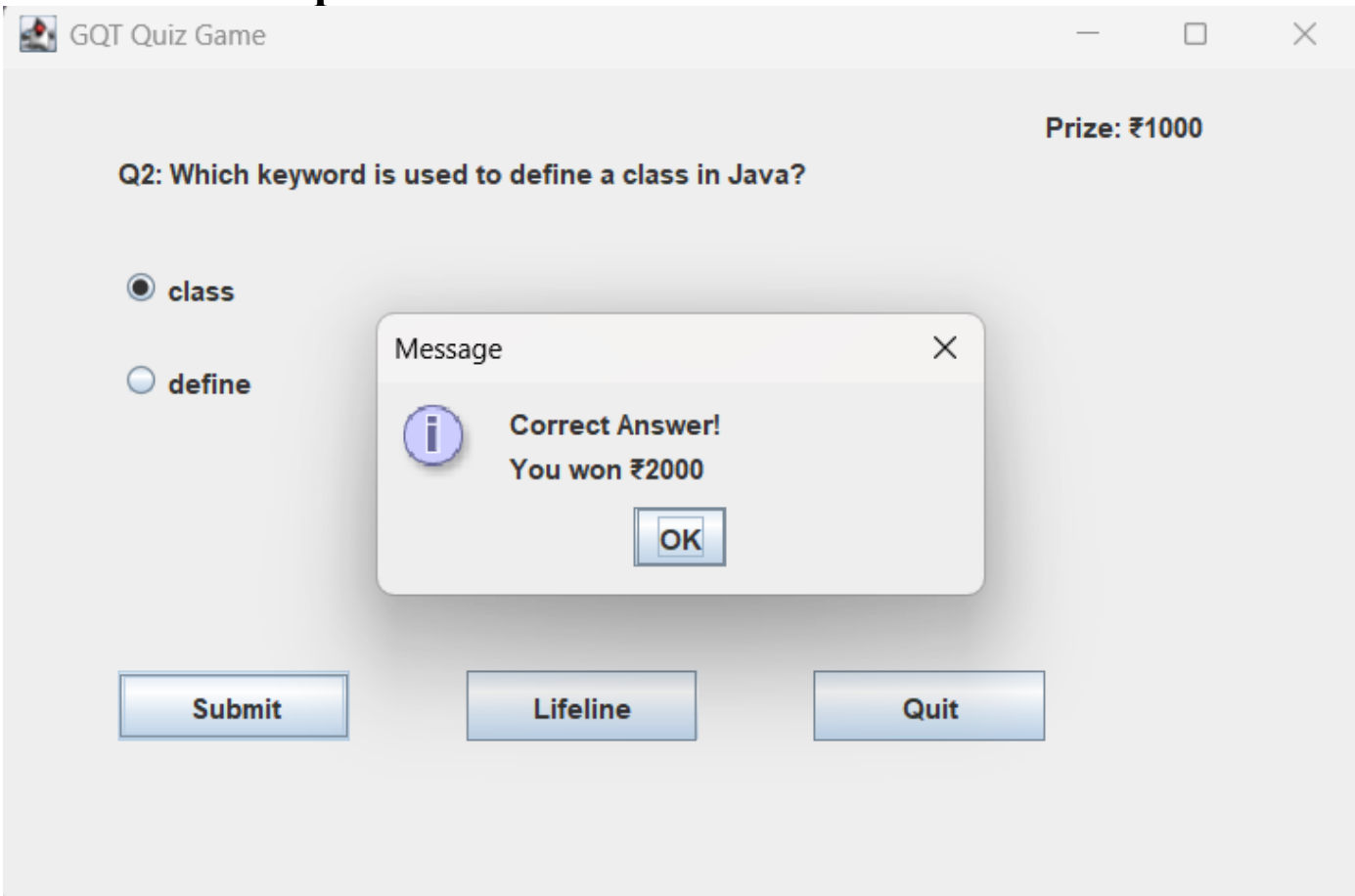
# //Choose Life Line



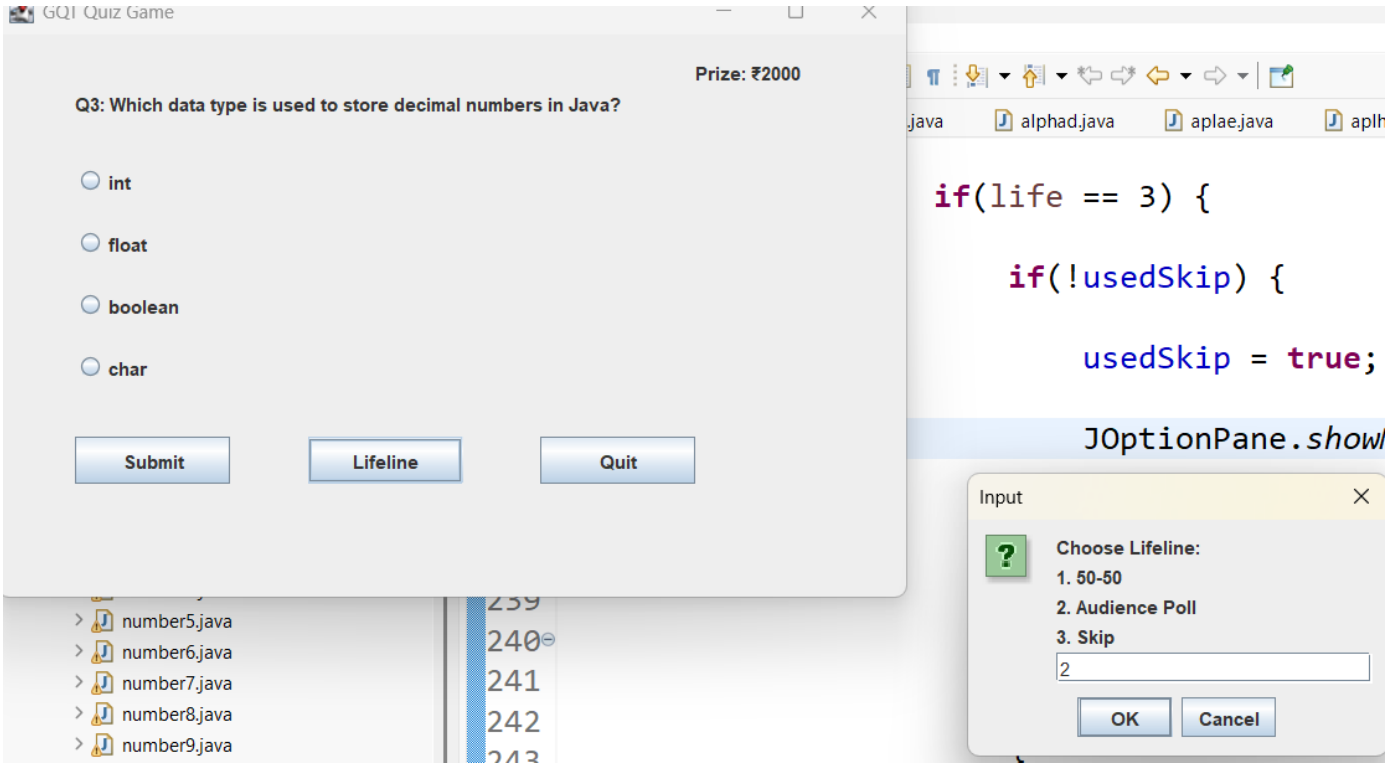

# //Life Line 50-50



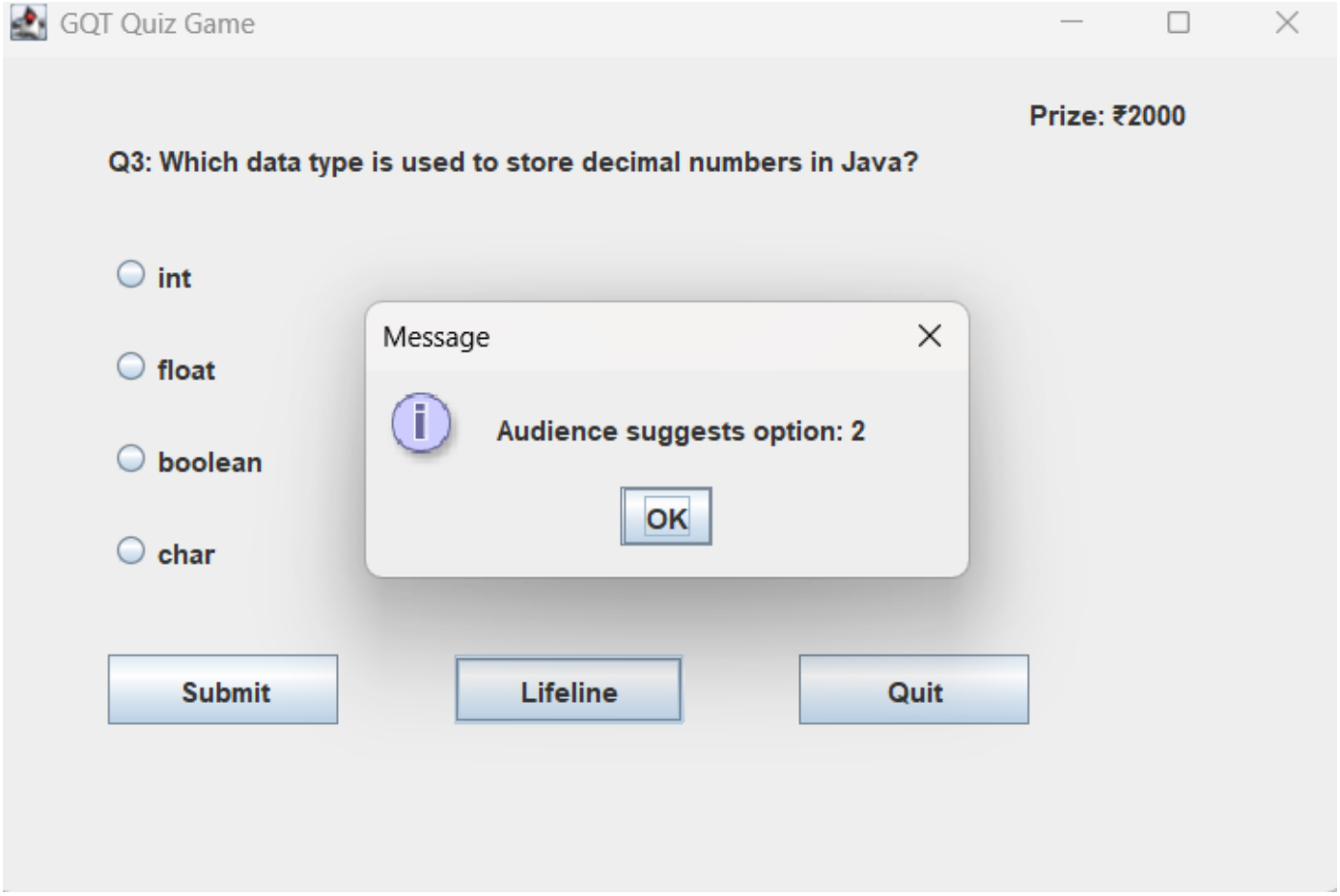
## //Choose The Option



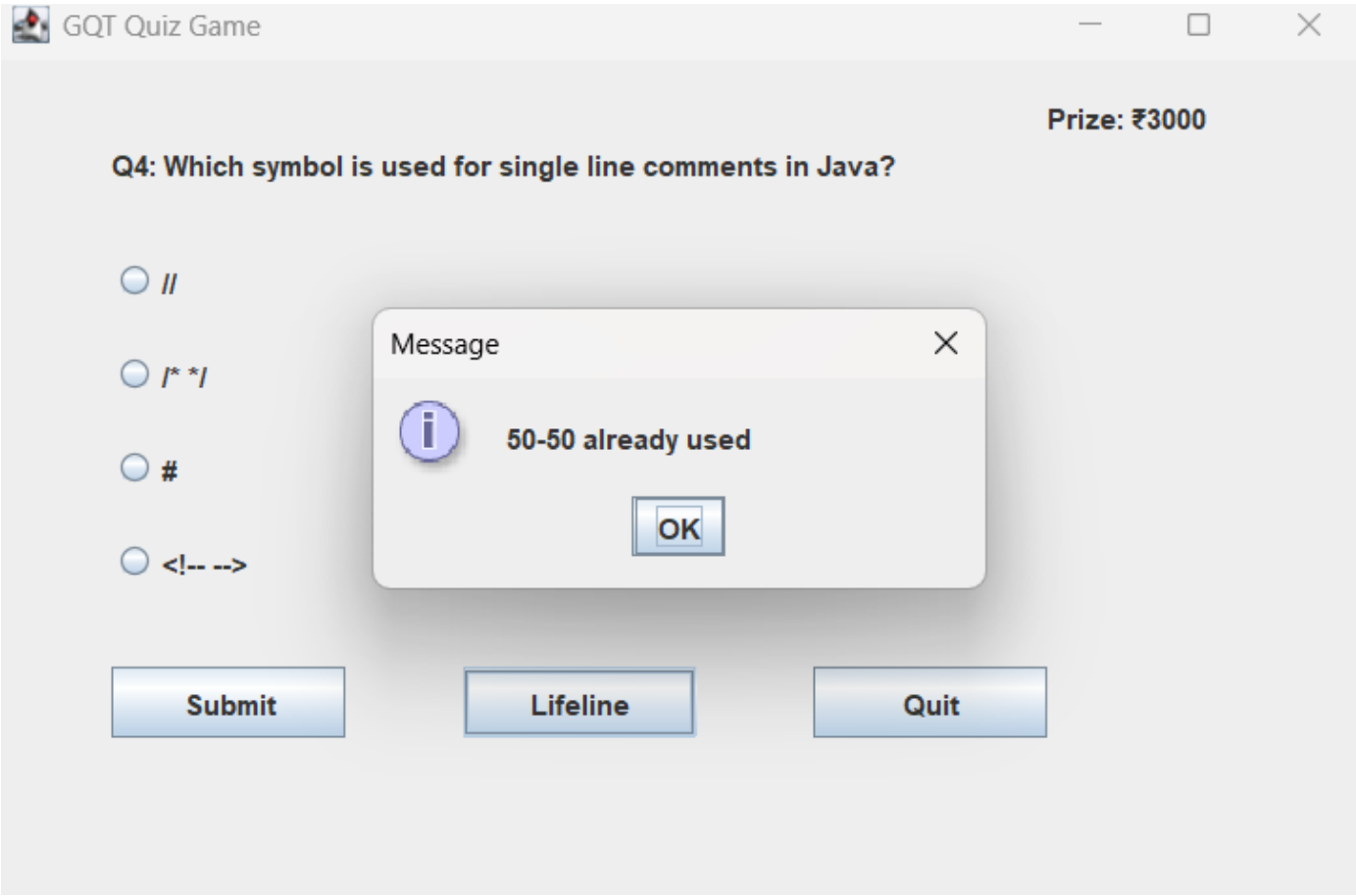

## //Life Line Audience Poll



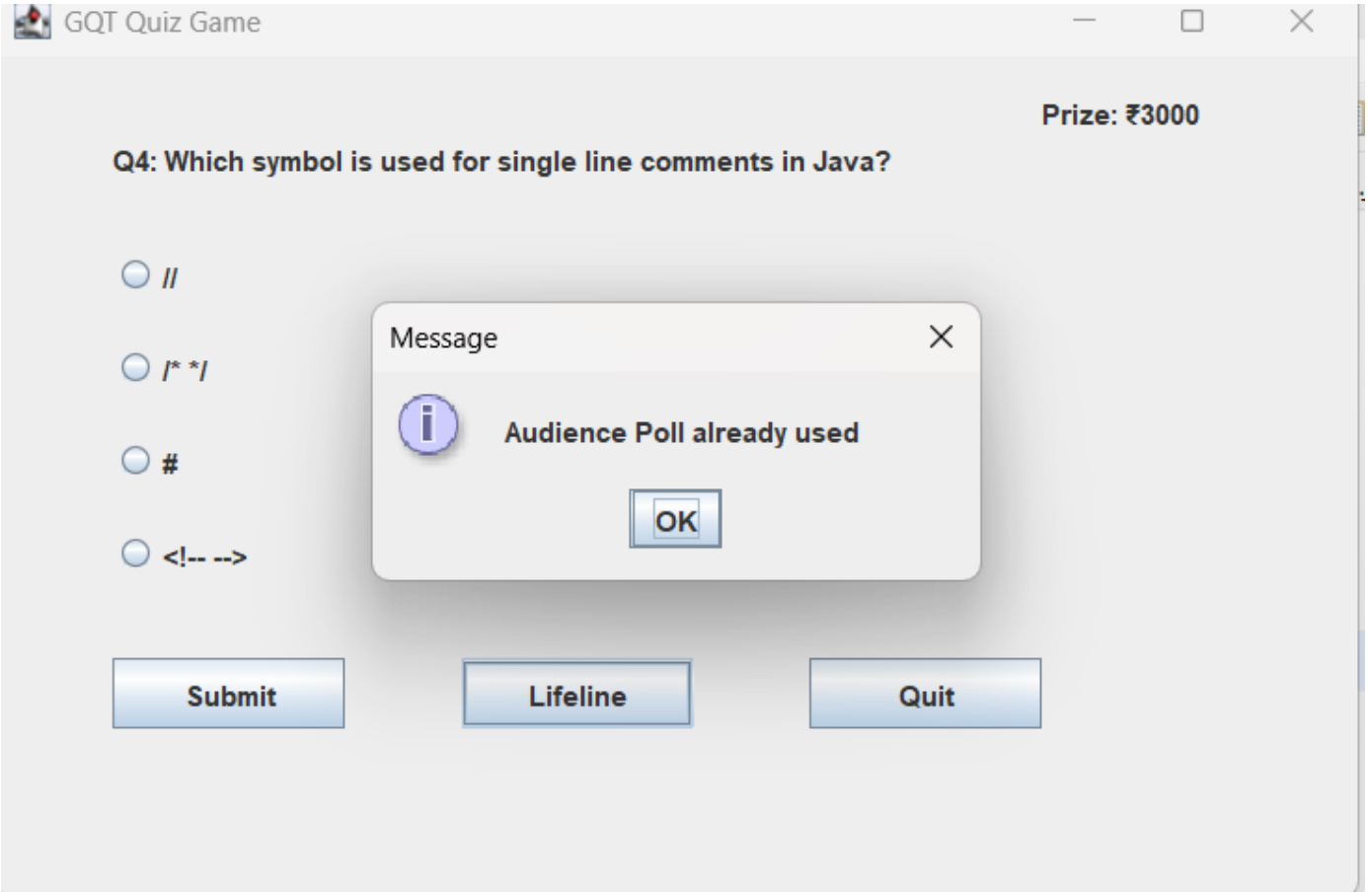
## //Audience Poll Activated



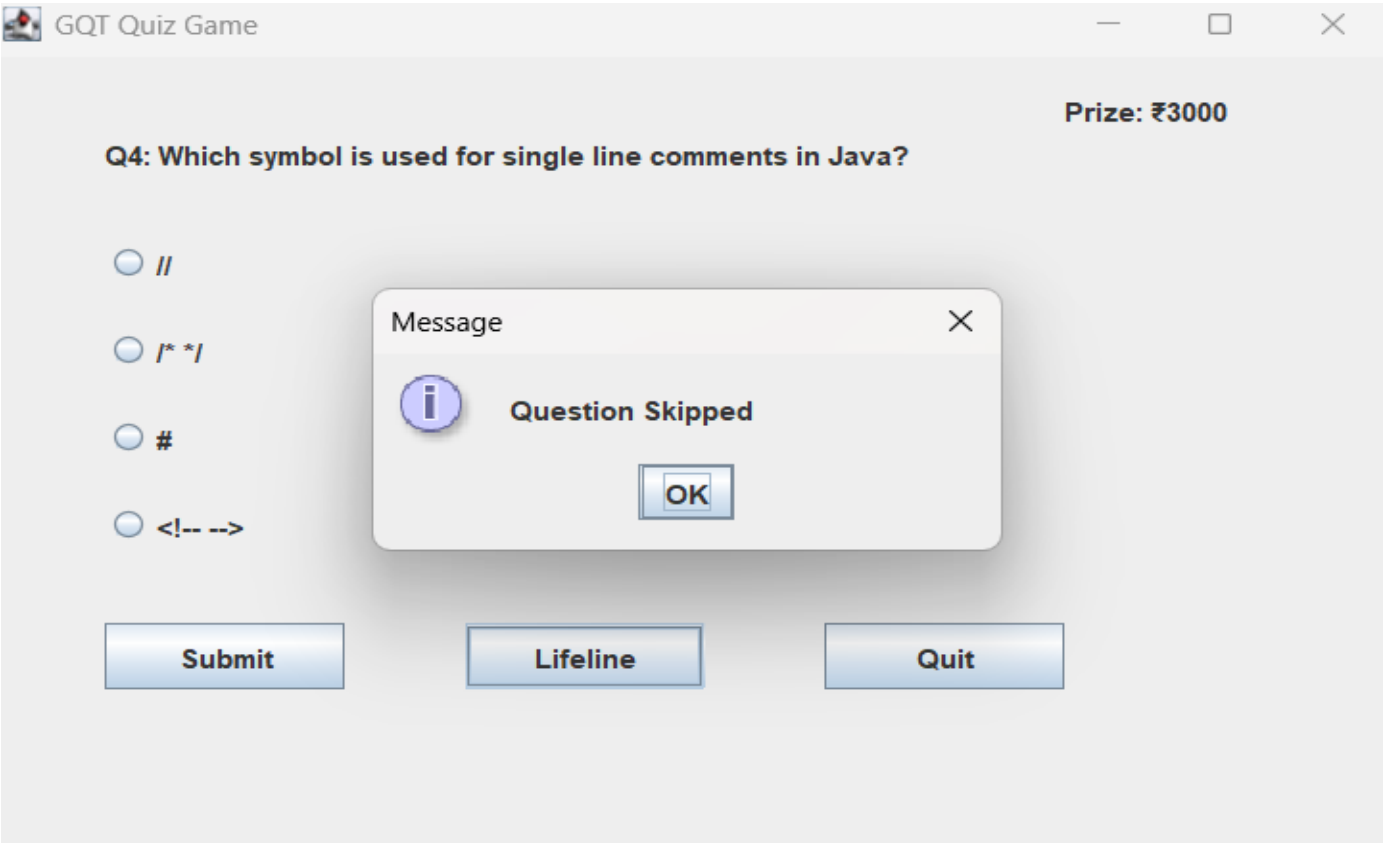

## //50-50 Poll Error



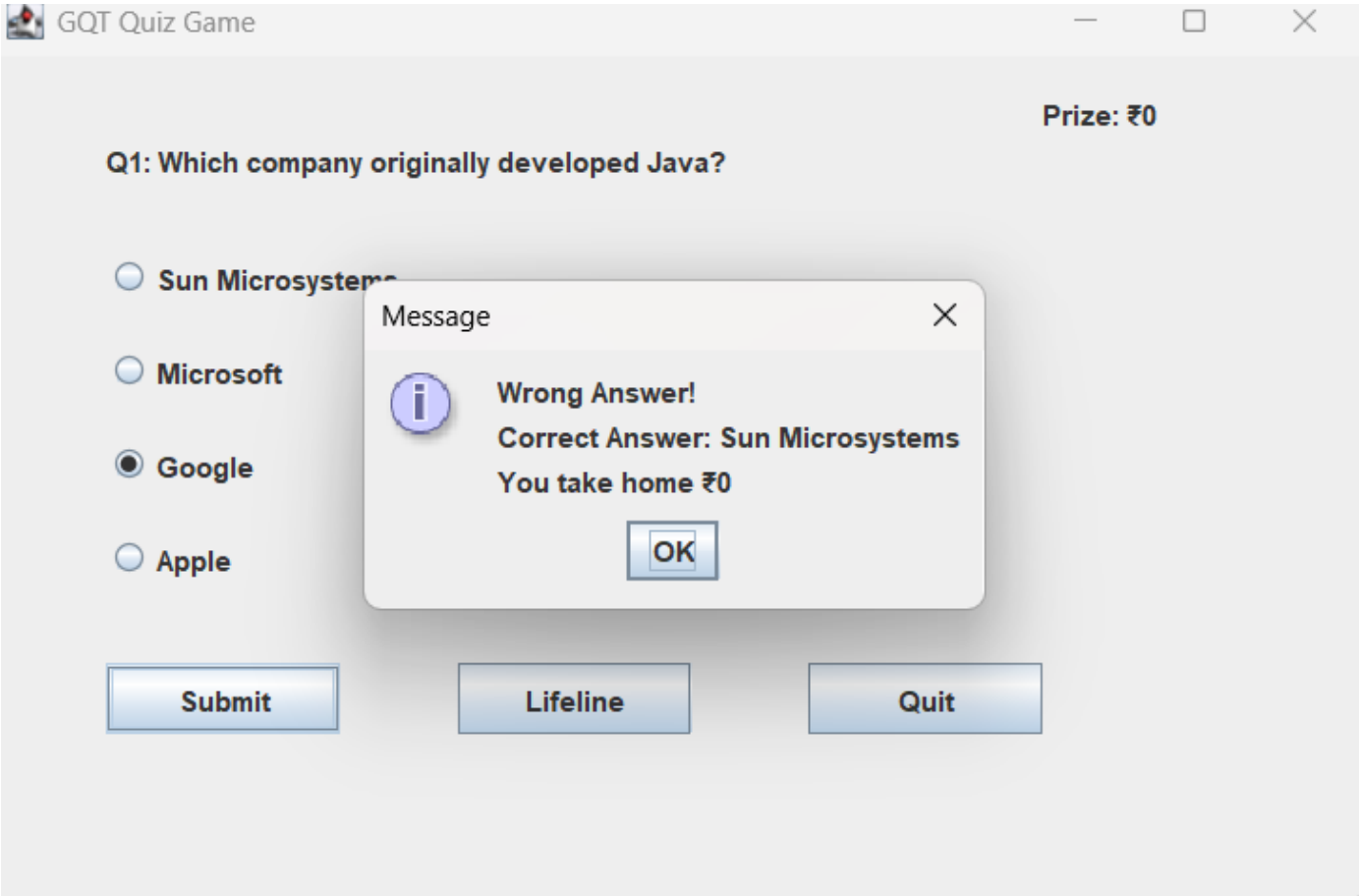
## //Audience Poll Error




## //Skipped Question



## //Wrong Answer




## //Quit The Game

## Conclusion:

In conclusion, the Java-Based Quiz Program using Swing is a great example of how core Java concepts can be combined with graphical user interface design to create an interactive and easy-to-use desktop application. By moving beyond a simple console-based quiz, this project provides a visually appealing and responsive interface that makes the quiz more enjoyable for users. It demonstrates the practical use of programming concepts such as arrays, loops, and conditional statements, while also showing how event handling works in a real application. The use of Swing components like buttons, labels, radio buttons, and dialog boxes makes the program organized and user-friendly. Features like lifelines, safe levels, a reward system, and instant feedback add an exciting, game-like experience, keeping users engaged and motivated. Moreover, options like start confirmation and quit functionality make the application more convenient and easy to control. Overall, this project not only improves logical thinking and problem-solving skills but also provides valuable experience in building graphical applications in Java. It lays a strong foundation for creating more advanced programs in the future, with better features and improved user interfaces.

www.gqtech.in
GLOBAL QUEST
TECHNOLOGIES

fuel your
**passion** for
**IT** with
our **guidance.**

# Global
# Quest Technologies

#324,2nd Floor, 3 A Cross, Near
Seshadripuram First Grade College,
Above City Union Bank,
Yelahanka New Town,
Bengaluru-560064
+91 9448 403 469 | 080-49720009
info@gqtech.in I www.gqtech.in